

For example 1: If the initial stack with size=5 is empty and the numbers 5, 10, 15, 20 are pushed in that order and popped 4 times, then 25, 30, 35 are pushed in that order and popped 2 times, what value will remain on the stack?

Answer: 25 (ค่าที่เหลืออยู่ใน Stack คือ [25], Top ชีที่ 25)

* Trace Ex 1: Push 5, 10, 15, 20 -> Pop 4 ครั้ง (20,15,10,5 ออก หมดคิ) -> Push 25, 30, 35 -> Pop 2 ครั้ง (35, 30 ออก) -> เหลือ [25]

For example 2: If the initial stack with size=5 contains the numbers [10, 20, 30, 40] , and is popped twice, then pushed 50, 60 in that order, then popped 3 times, what value will remain on the stack?

Answer: 10 (ค่าที่เหลืออยู่ใน Stack คือ [10], Top ชีที่ 10)

* Trace Ex 2: เริ่มต้น [10, 20, 30, 40] -> Pop 2 ครั้ง (40, 30 ออก) -> Push 50, 60 [10, 20, 50, 60] -> Pop 3 ครั้ง (60, 50, 20 ออก) -> เหลือ [10]

For example 3: if there is no stack's instance variable before, write statements in main program to create the instance variable mystack with size=5. Then push the number 1, 2, 3, 4, 5 in that order, and pop 3 times by using class Stack's methods that we learned.

Finally write the element that remain on the stack's instance variable mystack.

[เฉลยเป็นทางการ] Ex 3 (Statements in main program & Remaining Element):

```
mystack = Stack(5)    # 1. ประกาศสร้าง Instance ใหม่เนื่องจากไม่มีตัวแปรมาก่อน (No stack before)
for i in range(1, 6): # 2. Push ค่า 1, 2, 3, 4, 5 ตามลำดับ เข้าสู่ mystack
    mystack.push(i)
for _ in range(3):    # 3. Pop ข้อมูลออก 3 ครั้ง (ดึงค่า 5, 4, 3 ออกไปตามลำดับ LIFO)
    mystack.pop()
-> The element that remain on mystack: [1, 2] (โดยมี Top ชีอยู่ที่ค่า 2 ขนาดคงเหลือ = 2)
```

Is Different from

For example 3.1: if there is the exist stack's instance variable mystack with size=5 contains the numbers [10, 20, 30, 40] ...

[ข้อควรระวังสำคัญ] สรุปหลักการ & ค่าเตือนจุดได้ 0 คะแนน (Why 3.1 is Different from 3):

1. ใน Example 3.1: มีอินสแตนซ์ mystack เดิมอยู่แล้ว และมีข้อมูลบรรจุอยู่แล้วคือ [10, 20, 30, 40]
2. [ค่าเตือน 0 คะแนน] ข้อควรระวังสูงสุดในห้องสอบ (จุดตัด 0 คะแนน):
ห้ามเขียนคำสั่ง mystack = Stack(5) ซ้ำในข้อ 3.1 เด็ดขาด!
เพราะการสร้างอินสแตนซ์ใหม่ จะไป Overwrite หัวตัวแปรเดิม ทำให้ข้อมูล [10, 20, 30, 40] หายไปทั้งหมด
3. แนวทางการตอบที่ถูกต้อง:
ต้องเรียกใช้เมธอดบนตัวแปร mystack เดิมทันที เช่น mystack.pop() หรือ mystack.push(...) โดยไม่ประกาศคลาสใหม่

[เปรียบเทียบโค้ด] เปรียบเทียบการทำงานระหว่าง Example 3 vs Example 3.1:

* แบบที่ 1 (Example 3 - No stack before):

```
mystack = Stack(5) # จำเป็นต้องประกาศเพื่อสร้างออบเจกต์ขึ้นมาใหม่ในหน่วยความจำ  
mystack.push(1); mystack.push(2); ...
```

* แบบที่ 2 (Example 3.1 - Exist stack [10, 20, 30, 40]):

ห้ามเขียน mystack = Stack(5) เด็ดขาด (มิฉะนั้นจะกลายเป็น Stack ว่าง ได้ 0 คะแนน)

ให้เขียนดำเนินการต่อจาก Stack เดิมทันที เช่น:

```
mystack.pop() # ดึง 40 ออก -> เหลือ [10, 20, 30]  
mystack.push(99) # ใส่ 99 เข้าไป -> กลายเป็น [10, 20, 30, 99]
```

[กฎข้อสอบ] กฎการทำข้อสอบเรื่อง Stack (LIFO - Last In First Out):

- * การเข้าถึงข้อมูล: ข้อมูลตัวล่าสุดที่ Push เข้าไป จะเป็นตัวแรกที่ถูก Pop ออกมาเสมอ (Top Pointer)
- * การเขียนคำตอบในสไลด์/ใบงาน: ให้ระบุภาพ Stack จากกันไปยังยอด เช่น [10, 20] พร้อมชี้บอกตำแหน่ง Top
- * ข้อระวังเรื่องขนาด (Capacity): เมื่อสร้าง Stack(size=5) จะจุได้สูงสุด 5 ช่อง หากเกินจะเกิด Stack Overflow